

SIMATS ENGINEERING

Department of Computer Science and Engineering ASSESSMENT TOOL2 -CONCEPT MAPPING

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO1	Assessment:	ASSESSMENT TOOL 2- CONCEPT MAPPING
Weightage:	35%	Total Marks:	25
CO1 Statement:	Analyze the functional units of a computer system including CPU, memory, bus structures, and I/O components by examining instruction execution cycles, addressing modes, and performance metrics such as CPI, clock cycles, and benchmarking(BL4)		
Student Name:	SARU HASINI.S	Reg.No.:	192521418
Department:	BTECH IT	Date:	22/07/26

ANNEXURE A

1. Construct a detailed concept map showing the relationship between CPU (ALU, Control Unit, Registers), memory (cache, RAM), and I/O devices. Extend the map to include single-bus, multi-bus, and hierarchical bus structures, and clearly illustrate how data and control signals flow among these components during execution. Indicate possible points of delay or bottlenecks and how different bus structures help in improving performance.
2. Develop a comprehensive concept map linking the instruction execution cycle (fetch, decode, execute) with CPU performance parameters such as CPI, clock cycles, and execution time. Show how each stage contributes to overall execution time, and include connections to factors like memory access, instruction complexity, and pipeline stalls. Also, represent how benchmarking is used to evaluate performance.
3. Create a concept map comparing 8085 and 8086 architectures, including their instruction sets, register organization, addressing modes, and memory segmentation (in 8086). Clearly show how these architectural features influence program execution, multitasking capability, and efficiency in real-time applications.

4. Draw a detailed concept map that connects various addressing modes (immediate, direct, register, indirect, indexed) with instruction types (data transfer, arithmetic, control instructions). Illustrate how each addressing mode affects memory access time, instruction size, and execution speed, and show real-world scenarios where specific addressing modes are preferred.

5. Prepare an elaborate concept map integrating number systems (binary and hexadecimal) with instruction representation, memory storage, and debugging processes. Show how data is converted between formats, how instructions are stored and interpreted by the CPU, and why hexadecimal representation is commonly used in low-level programming and system design.

Code of Conduct:

I SARU HASINI.S (192521418) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Saruhasini.s

MARKS ALLOCATION

Criteria	Concept Identification (25%)	Linkages (25%)	Organization (20%)	Integration of Literature (20%)	Clarity (10%)
Q1					
Q2					
Q3					
Q4					
Q5					

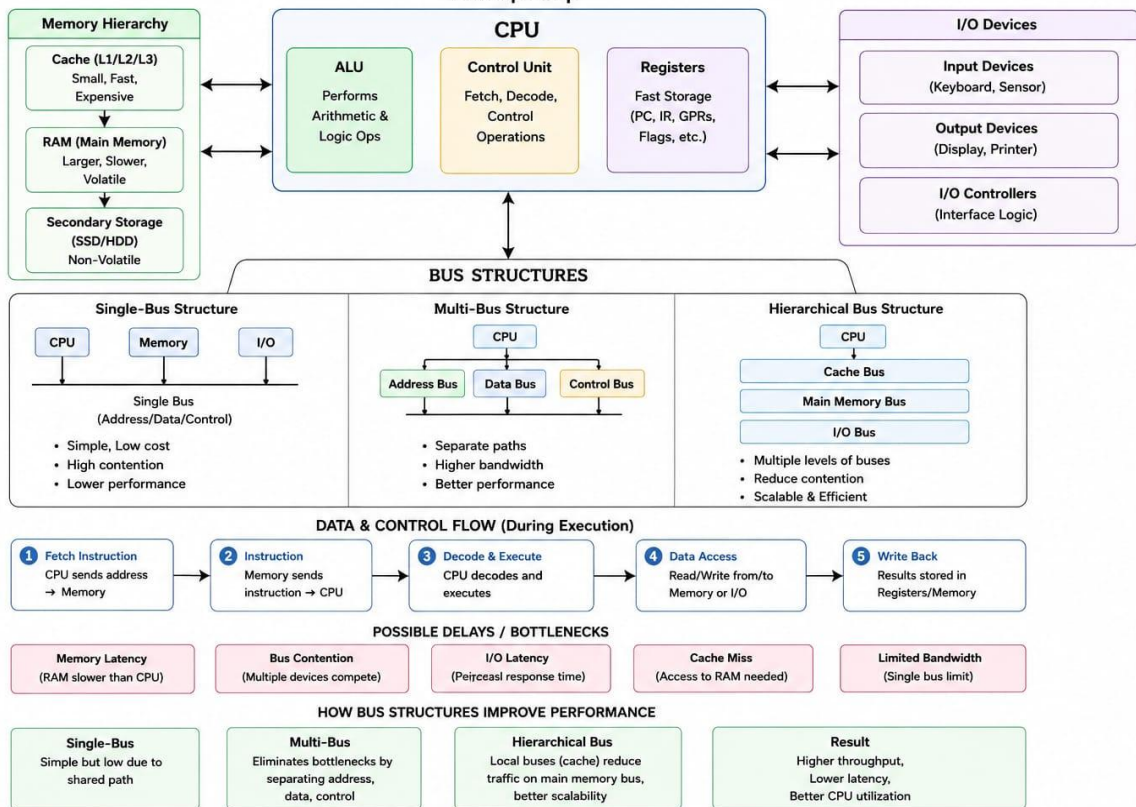
RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Concept Identification	25%	Identifies all key concepts from literature accurately and comprehensively	Identifies most key concepts with minor omissions	Identifies limited concepts; some are irrelevant or missing	Fails to identify essential concepts
Linkages	25%	Establishes clear, meaningful, and accurate relationships between concepts	Shows correct relationships with minor gaps	Relationships are weak, unclear, or partially incorrect	No meaningful relationships established
Organization	20%	Concepts are well structured hierarchically with logical flow and grouping	Mostly organized with minor structural issues	Some organization but lacks clear hierarchy	No logical organization or structure
Integration of Literature	20%	Integrates multiple studies effectively to show connections and comparisons	Shows some integration of literature	Limited integration; mostly isolated concepts	No integration of literature evidence
Clarity	10%	Concept map is clear, neat, visually structured, and easy to interpret	Generally clear with minor readability issues	Clarity is reduced; difficult to interpret in parts	Poorly presented and hard to + understand

1

CPU, MEMORY, I/O & BUS STRUCTURES

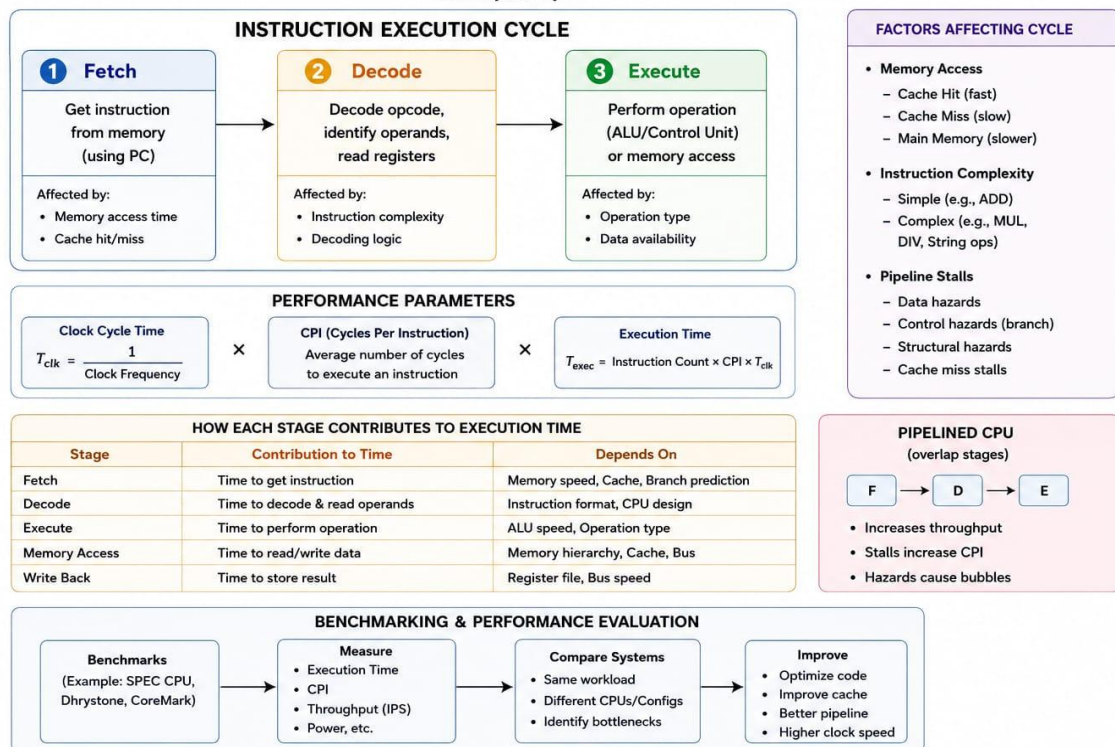
Concept Map



2

INSTRUCTION EXECUTION CYCLE & CPU PERFORMANCE

Concept Map



3

8085 vs 8086 ARCHITECTURE COMPARISON

Concept Map

	8085 (8-bit Microprocessor)	8086 (16-bit Microprocessor)
Introduction	Introduced by Intel in 1976	Introduced by Intel in 1978
Architecture	8-bit	16-bit
Data Bus	8-bit	16-bit
Address Bus	16-bit (64 KB memory)	20-bit (1 MB memory)
Registers	6 General Purpose (B, C, D, E, H, L) + 2 Special (PC, SP) + 1 Flag Register	8 General Purpose (AX, BX, CX, DX, SI, DI, BP, SP) + 4 Segment Registers (CS, DS, SS, ES) + IP, Flag Register
Instruction Set	74 Instructions	~120 Instructions
Addressing Modes	5 (Immediate, Register, Direct, Register Indirect, Implied)	12 (Immediate, Register, Direct, Register Indirect, Based, Indexed, Based Indexed, Relative, etc.)
Memory Segmentation	No Segmentation	Yes (CS, DS, SS, ES segments) Logical Address = Segment : Offset (16 bits) : (16 bits) Physical Address = (Segment × 16) + Offset → 1 MB Address Space
Multiprocessing / Multitasking	Limited (No protection, single task at a time)	Better (Segmentation allows code, data, stack separation; supports multitasking OS)
Performance in Real-time Applications	Good for simple control tasks, small memory programs	Better for complex tasks, larger programs, faster data processing

IMPACT ON PROGRAM EXECUTION & EFFICIENCY

Larger Address Space (8086) → More data & code can be handled

Segmentation (8086) → Memory Protection & Multitasking

More Registers & Addressing Modes (8086) → Efficient Code

16-bit Data Path (8086) → Higher Throughput

Overall: 8086 is more powerful, scalable, and efficient than 8085.

4

ADDRESSING MODES & INSTRUCTION TYPES

Concept Map

ADDRESSING MODES					INSTRUCTION TYPES			
Addressing Mode	Syntax / Notation	Operand is...	Used For / Example	Pros / Cons	Type	Opcode / Symbol	Performs	Example
Immediate	#data	Operand is part of the instruction	• Fast (no memory access) Example: MOV A, #5	• Fast • Small data only	Data Transfer	MOV, LDA, STA, XCHG, PUSH, POP	Move data between registers, memory, or I/O	MOV A, R1 LDA 2000H
Direct	addr	Address of operand is given in instruction	• One memory access Example: MOV A, 2000H	• Simple • Limited address range	Arithmetic	ADD, SUB, ADC, SBB, INR, DCR	Arithmetic operations (add, subtract, increment, etc.)	ADD A, B SUB A, #10
Register	R	Operand is in CPU register	• Fastest (no memory access) Example: MOV A, R1	• Very fast • Limited by number of registers	Control	JMP, JZ, JC, CALL, RET	Program flow control (branching, calls, returns)	JZ LOOP CALL SUBR
Register Indirect	(R)	Register holds the memory address of the operand	• One memory access Example: MOV A, (R1)	• Flexible • Extra memory access	Logical	ANL, ORL, XRL, CPL, CLR, SETB	Logical operations (AND, OR, XOR, complement, set)	ANL A, B CPL C
Indexed	addr(R)	Effective address = address + index register	• One memory access Example: MOV A, ARRAY(R1)	• Flexible for arrays • Good for looping	I/O	IN, OUT	Input from or output to I/O ports	IN 80H OUT 90H

EFFECT ON PERFORMANCE			
Addressing Mode	Memory Accesses	Instruction Size	Execution Speed
Immediate	None	Small	Fastest
Direct	1 Memory Access	Medium	Fast
Register	None	Small	Fastest
Register Indirect	1 Memory Access	Medium	Fast
Indexed	1 Memory Access	Larger	Moderate

Register & Immediate modes are fastest; Indexed is flexible but slightly slower.

REAL-WORLD SCENARIOS (Where Preferred)		
Type	Best Suited For	Why?
Immediate	Constants, flags, counters	No memory access, quick execution
Direct	Global variables, I/O mapped data	Simple access to fixed addresses
Register	Frequent operations, temporary data	Fastest access, used by ALU
Register Indirect	Pointers, dynamic data structures	Address in register can change
Indexed	Array processing, table lookup	Easy traversal using index register

KEY TAKEAWAYS

- Different addressing modes trade off speed, flexibility, and instruction size.
- Instruction types define what operation the CPU performs.
- Choosing the right mode & instruction improves performance.

LEGEND

- Data Movement
- Arithmetic
- Control
- Logical
- I/O

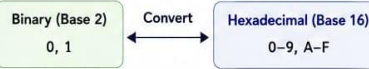
TIP

- Use registers whenever possible for speed.
- Use immediate for constants.
- Use indexed for arrays and tables.
- Use direct for fixed memory locations.

NUMBER SYSTEMS, INSTRUCTION REPRESENTATION, MEMORY STORAGE & DEBUGGING

Concept Map

1 NUMBER SYSTEMS



4-bit Binary $\leftrightarrow$ Hex

Binary	Hex
0000	0
0001	1
...	...
1001	9
1010	A
1011	B
...	...
1111	F

Why Hex?

- Compact (4 bits = 1 hex digit)
- Easy for humans to read
- Matches memory & address representation

2 INSTRUCTION REPRESENTATION

Instruction Format (Example: 8085)

Opcode (8 bits) Operand (8/16 bits)

Example: MVI A, 25H

Binary : 0011 1110 0010 0101
Hex : 3E 25

- CPU fetches opcode $\rightarrow$ decodes
- Fetches operand (if any)
- Executes the instruction

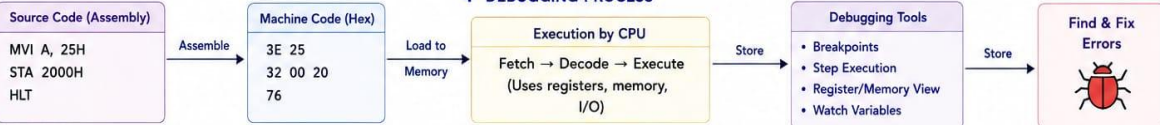
3 MEMORY STORAGE

Memory is a linear array of bytes

Address (Hex)	Data (Hex)	Data (Binary)
2000H	3E	0011 1110
2001H	25	0010 0101
2002H	32	0011 0010
2003H	00	0000 0000
...	...	...

- Each address holds 8 bits (1 byte)
- Multi-byte data stored in consecutive addresses (Little Endian)

4 DEBUGGING PROCESS



HOW DATA IS CONVERTED BETWEEN FORMATS



WHY HEX IS COMMON IN LOW-LEVEL PROGRAMMING

- Matches hardware (nibbles, bytes, words)
- Compact and unambiguous representation
- Easier debugging and memory inspection
- Widely used in assembly, machine code, device registers, and protocols

KEY TAKEAWAYS

- Binary is the foundation; Hex makes it human-friendly.
- Instructions are encoded as opcode + operand.
- Memory stores data as bytes at unique addresses.
- Debugging helps detect and fix errors efficiently.

LEGEND (Color Guide)

